Computer Systems Literacy

MODULE 3: SYSTEM ARCHITECTURE & DATA FLOW

Agenda – Topics to be Covered

Von Neumann vs. Harvard Architecture: Understanding the shared path for data and instructions.

The Memory Hierarchy: Why we need L1/L2/L3 caches, RAM, and Swap Space.

The Bus & Throughput: Understanding bandwidth, clock speeds, and the "Bottleneck Effect."

I/O Latency: The massive time scale difference between a CPU cycle and a disk read.

Project Mercy

Defining Architecture

Definition 1: Architecture is the enduring structural blueprint of a system that governs how its parts are organized, interact, and evolve.

Architecture answers:

1. What are the core components?
2. How do they communicate?
3. Where are responsibilities separated?
4. What assumptions and trade-offs are locked in?

Architecture **matters** because it constrains what is possible, determines efficiency and scalability, encodes assumptions about the world, and is hard to change once deployed.

Project Mercy

Types of Architecture by Domain

Meaning of architecture according to domain:

1. **Machine / Hardware Architecture**: Physical and logical organization of hardware.
   - Example: CPU architecture (x86, ARM), memory hierarchy, I/O pathways and buses.
   - The architecture defines the instruction set, data paths, parallelism, performance ceilings.
2. **Computer / System Architecture**: How hardware and software are organized together.
   - Examples: Von Neumann vs. Harvard architecture, client-server[1], microservices vs. monolith, distributed systems[2].
   - Defines process boundaries[3], communication models[4], fault tolerance[5], scalability characteristics[6].
3. **Software Architecture**: Fundamental, high-level structural organization of software, acting as a blueprint for code and services.
   - Examples: Model-View-Controller (MVC[7]), layered architecture[8], event-driven systems[9].
   - Defines module boundaries[10], dependency directions[10], data flow, and changeability & maintainability.

Project Mercy

Von Neumann Architecture

There are 2 primary ways we design how a computer's "brain" (the CPU) talks to its "memory". First is the Von Neumann Architecture and the second is Harvard Architecture.

Von Neumann Architecture: This is the blueprint for most general-purpose computers (like your laptop). It uses a **single shared memory** and a **single bus** (pathway) for both program instructions and data.

- The Pro: It's flexible and simple to design.
- The Con: The "Von Neumann Bottleneck". Because there is only one path, the CPU cannot fetch an instruction and read/write data at the exact same time.

CPU Cycle Pattern (Simplified): Fetch Instruction > Decode > Fetch Data > Execute > Write back to Memory.

Von Neuman Architecture

1. **Stored-Program Concept:** Unlike early machines (which were hard-wired), modern computers store both *Instructions* (code) and *Data* (variables) in the same memory unit.
2. **The Components of a Von Neuman Computer:**
    - **CPU (Control Unit + ALU):** Performs the math and directs traffic.
    - **Memory Unit:** Holds data/instructions.
    - **I/O:** Interfaces with the outside world.
3. **The Von Neumann Bottleneck**
    1. Because **instruction fetch** and **data access** share the same path, they cannot happen simultaneously.
    2. The CPU is incredibly fast, but the connection to Memory (the "Bus") is limited. The CPU spends most of its life *waiting* for instructions and data. This bottleneck is architectural, not just technological.

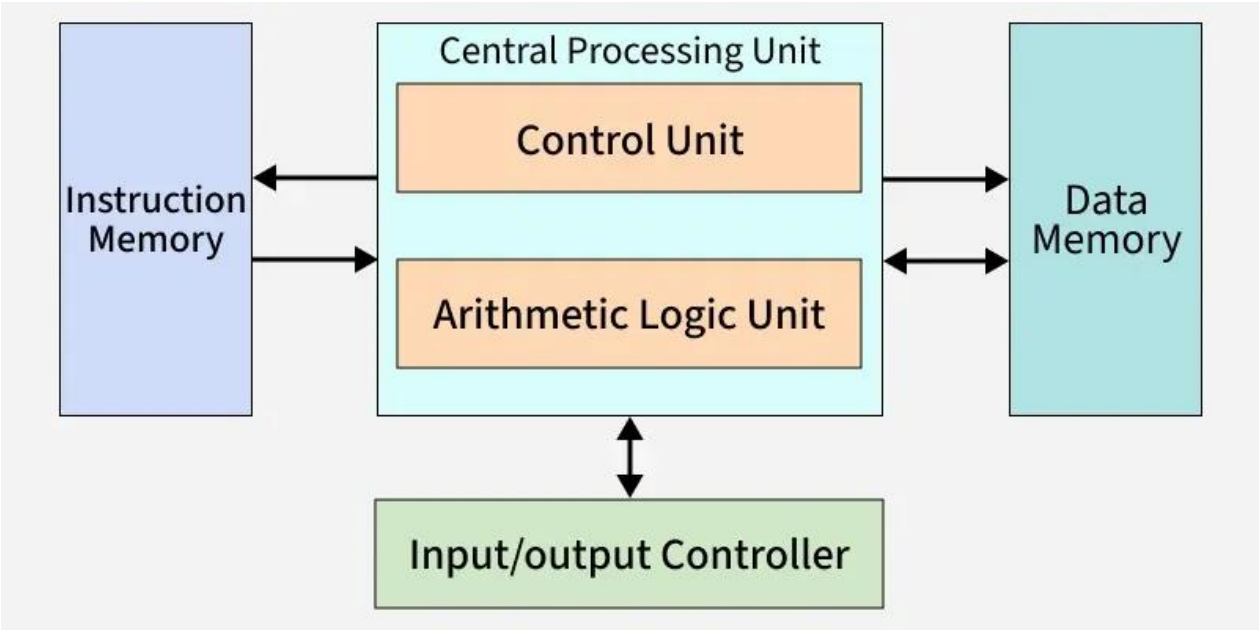
Harvard Architecture

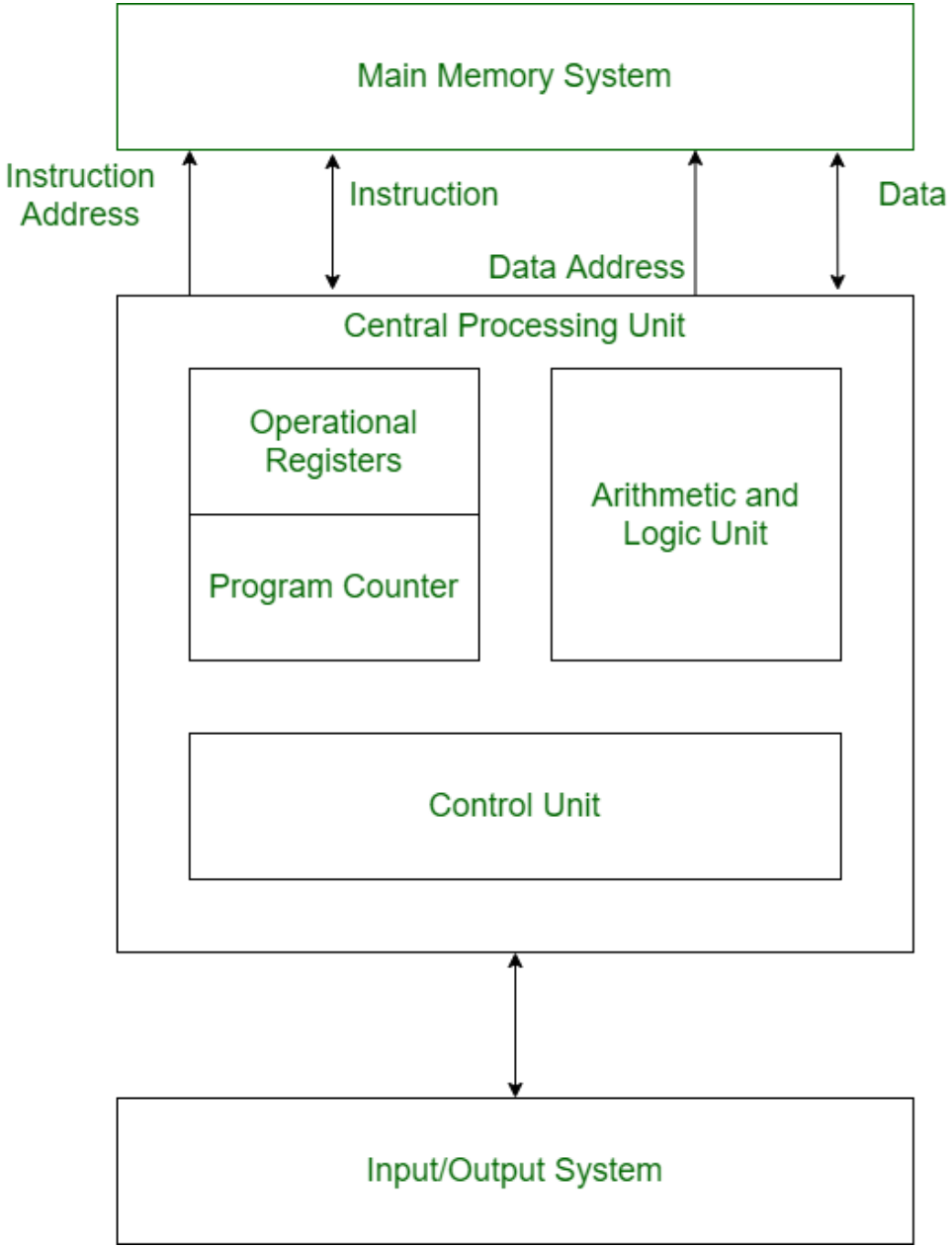
Harvard architecture is a computer design model where program instructions and data are stored in **separate memory units** that are accessed through **independent buses**. This separation allows the processor to fetch instructions and access data simultaneously, which helps avoid the bottleneck present in traditional Von Neumann systems. Common in specialized microcontrollers and Signal Processors (DSP).

The Pro: The CPU can read an instruction and access data simultaneously, leading to higher performance for specific tasks. Eliminates the Von Neumann Bottleneck. Using **parallel access,** the CPU can fetch next instruction *while* current instruction accesses data.

The Con: More complex hardware and less flexibility in how memory is used.

Harvard Architecture





Harvard Architecture

In Harvard architecture, instruction and data paths are physically separated. This separation shows as distinct buses, each with a specific role.

1. **Data Bus**: Carries data between the CPU and the data memory. Supports read and write operations.
2. **Data Address Bus**: Carries the address of data to be read/written from the CPU to the memory.
3. **Instruction Bus**: Carries instructions from program (instruction) memory to the CPU. Read-only in most designs, meaning the CPU can only read from the instruction memory & bus is unidirectional.
4. **Instruction Address Bus**: Carries the address of instructions to be fetched from instruction memory.

Often included in practical implementation:

1. **Control Bus**: Control signals like read/write, enable, clocking, interrupts. Sometimes shared, sometimes split into instruction-side and data-side control lines.
2. **I/O Bus**: Used for communication with peripherals. In pure Harvard, I/O may hang off the data bus.

Modern Computing

Modified Harvard (Modern CPUs):

- **Separate L1 instruction cache (I-cache) and data cache (D-cache)**
- Unified L2/L3 caches

Gives Harvard-style parallelism **without duplicating main memory**

Why Von Neumann Still Exists

- Simpler programming model
- Flexible (code can modify data, data can become code)
- Essential for general-purpose computing

Key insight: Von Neumann is simple but bandwidth-limited. Harvard is fast but rigid. Modern CPUs hybridize both.

Memory Hierarchy

The goal of the memory hierarchy is to create an illusion of a memory pool that is as fast as the most expensive memory and as large as the cheapest.

1. **L1, L2, and L3 Caches**: These are tiny amounts of ultra-fast memory located directly on the CPU chip.
   - L1: The fastest and the smallest (kilobytes).
   - L2 is slightly larger and slower.
   - L3 is shared across CPU cores. They exist because the CPU works much faster than RAM can keep up with.
2. **RAM (Random Access Memory)**: The "workspace" of the computer. It is much larger than cache but significantly slower. If the CPU can't find what it needs in the caches ("a cache miss"), it goes to RAM.
3. **SWAP Space (Virtual Memory)**: When your RAM is full, the operating system uses a portion of your hard drive (SSD/HDD) as "overflow" memory. **Warning:** Because even the fastest SSD is thousands of times slower than RAM, using swap space causes a massive performance drop (often called "thrashing").

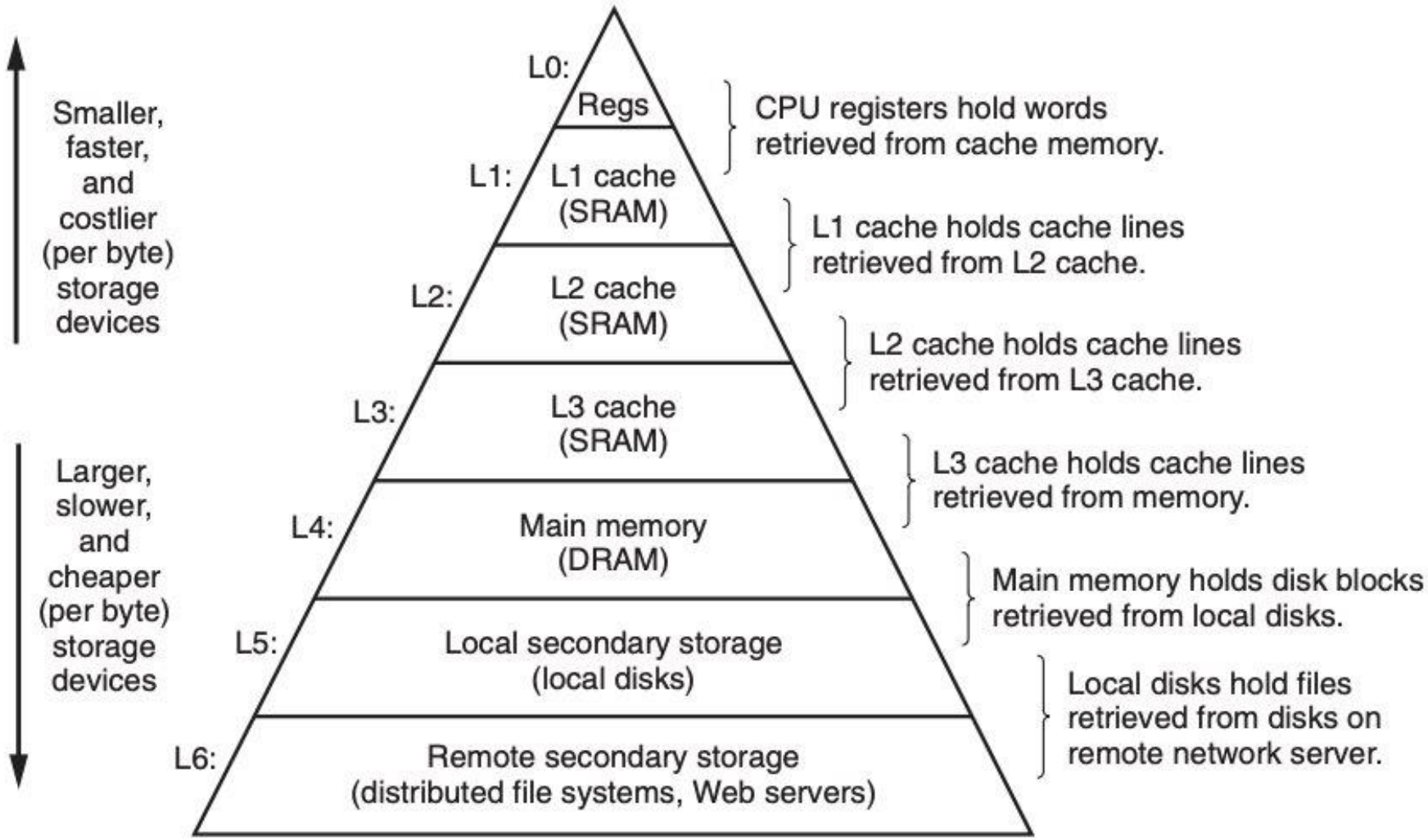


Figure 1.9 **An example of a memory hierarchy.**

Core Idea: Fast memory is expensive, cheap memory is slow.

Caching

Definition: Caching is storing frequently used data in a temporary, high-speed storage (cache) to deliver it faster, improving performance by reducing reliance on slower primary data sources like databases or disks.

<u>The CPU and L1 Cache: The "Internal Bus"</u>

- The connection of the CPU to the **L1 Cache** is the fastest "bus" in the entire computer.
- **Location:** It is entirely inside the CPU core.
- **Speed/Width:** Because L1 cache needs to feed the CPU instantly, this bus is often much wider than the 64-bit system bus (sometimes 256-bit or 512-bit) to support massive instructions like AVX-512. It runs at the full clock speed of the CPU (e.g., 5.0+ GHz).

Caching

When the CPU needs a piece of data, it starts a high-speed search relay.

1. **L1 Cache**: The CPU first looks in the **L1 Cache**. Because it is split into **L1i** (Instructions) and **L1d** (Data), the CPU can grab "what to do" and "what to do it to" simultaneously. If it's here, it's a **"Cache Hit"** and work continues at full speed.
2. **L2 Check (The Desk):** If L1 misses, the CPU checks **L2**. L2 is the "middle manager." It is larger and holds bigger chunks of code that might be needed in a few milliseconds.
3. **L3 Check (The Shared Office):** If L2 misses, the core looks in the **L3 Cache**. This is critical because L3 is **shared**. If Core 1 already loaded a texture into L3, Core 2 can grab it from there instead of going to the slow RAM.
4. **The RAM Trip (The Grocery Store):** If it's not in L3, the CPU must stop and wait. It sends a request to the RAM. In CPU time, this feels like an eternity—about 200–300 cycles of doing nothing.

Caching: The Rules of Cooperation

The caches don't just store random data; they follow strict "inclusion" rules to stay in sync.

Inclusive vs. Exclusive Policies

- **Inclusive Cache:** L3 contains everything that is in L2, and L2 contains everything in L1. If you delete something from L3, the hardware automatically clears it from L1 and L2 to keep things simple. (Common in Intel chips).
- **Exclusive Cache:** Data exists in **only one** level. If the CPU moves data from L2 to L1, it is deleted from L2 to save space. This effectively gives you "more" total cache. (Common in AMD Ryzen chips).

Caching: The Rules of Cooperation

Why this matters in 2026

- In modern gaming and AI workloads, we now see **3D V-Cache** (like on AMD's X3D processors). This involves physically stacking a massive block of extra L3 cache directly on top of the CPU.
- This allows the L3 to act as a "Gigantic Safety Net." Instead of the CPU "missing" and going to RAM 20% of the time, a huge L3 might catch 99% of those requests.
- **Result:** This removes the stuttering you feel in games when the CPU is forced to wait for the RAM to catch up during a complex scene.
- **Pro-Tip:** If you look at **Task Manager** (Performance Tab > CPU), you can see the exact sizes of your L1, L2, and L3 caches. Notice how the L3 is significantly larger than the others!

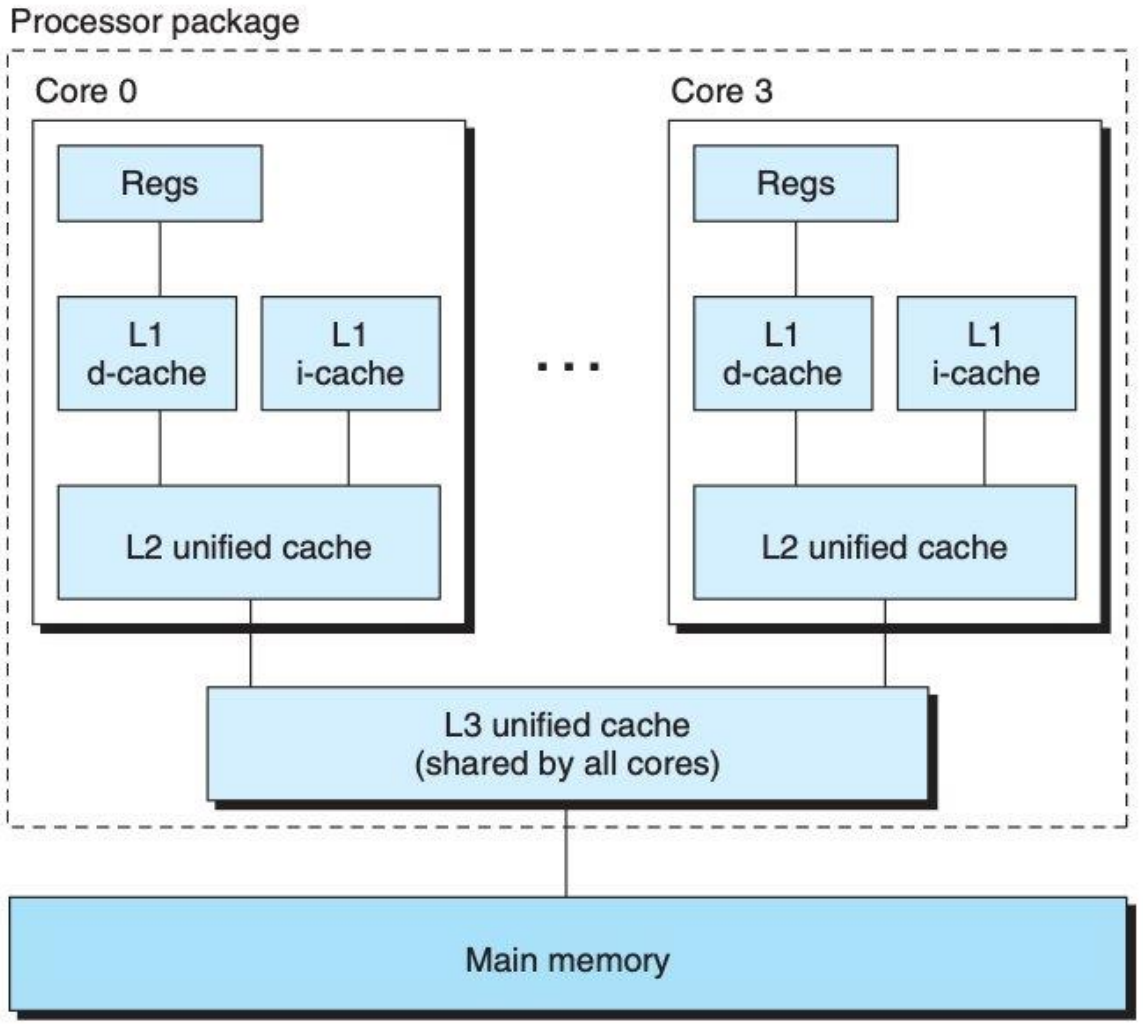
Caching: Pre-fetching & Promotion Policy

To understand **Pre-fetching** and the **Promotion Policy** (moving data up the chain), we have to look at the CPU as a "predictor" rather than just a "calculator."

1. **Pre-Fetching**: Pre-fetching is the process of fetching data from a slow area (RAM) to a fast area (Cache) **before** the CPU actually asks for it. It exploits the **Principle of Locality.** Caches work only because programs exhibit locality.
   a. **Spatial Locality:** If you ask for the first item in an array, you're almost certainly going to ask for the second item next. The pre-fetcher grabs a whole "block" of nearby data.
   b. **Temporal Locality:** If you just used a variable, you'll probably use it again in a few milliseconds.
2. **Moving Data Up**: Data isn't moved because it is frequent; it is moved because it was requested, and it stays there because it is frequent.
   a. If the data is in L3, it is immediately copied/moved up to L2, and then to L1 so that the CPU core can actually use it. The CPU Core cannot work on data while it is sitting in L3. It must be in the registers/L1 to be processed.

Cache Architecture in CPU

Figure 1.17
Intel Core i7 organization. Four processor cores are integrated onto a single chip.



Memory Hierarchy Latency

Level	Typical Latency	Why it exists
Registers	~1 CPU cycle	Operands for ALU
L1 Cache	~3–5 cycles	Immediate locality
L2 Cache	~10–15 cycles	Buffer for L1 misses
L3 Cache	~30–50 cycles	Shared among cores
RAM	~100–300 cycles	Main working memory
Swap (Disk)	Millions of cycles	Emergency overflow

Bus Systems & Throughput: Executive Summary

Concept: Buses are the wires connecting the various components.

Core Idea: How fast data can move between components.

- **Width vs. Speed:**
  - **Bus Width:** How many parallel wires are in each bus (e.g., 64-bit systems have 64 wires).
  - **Clock Speed:** The "speed limit" or the cadence at which data is pushed through the bus (e.g., 3200 MHz).
- **Throughput Formula:** Bus Width x Clock Frequency = Bandwidth. Eg: 64-bit bus × 3 GHz = 192 Gbps (theoretical max)
- **Modern Architecture:** The **Memory Controller** is now inside the CPU (integrated) to reduce latency, while the **Chipset** (PCH) handles slower traffic like USB and Audio.
- **The Bottleneck Effect:** This occurs when one component's maximum throughput is significantly lower than the others. For example, if you have a top-tier CPU but a slow, narrow bus, the CPU will spend most of its time "idling" while waiting for data to arrive.

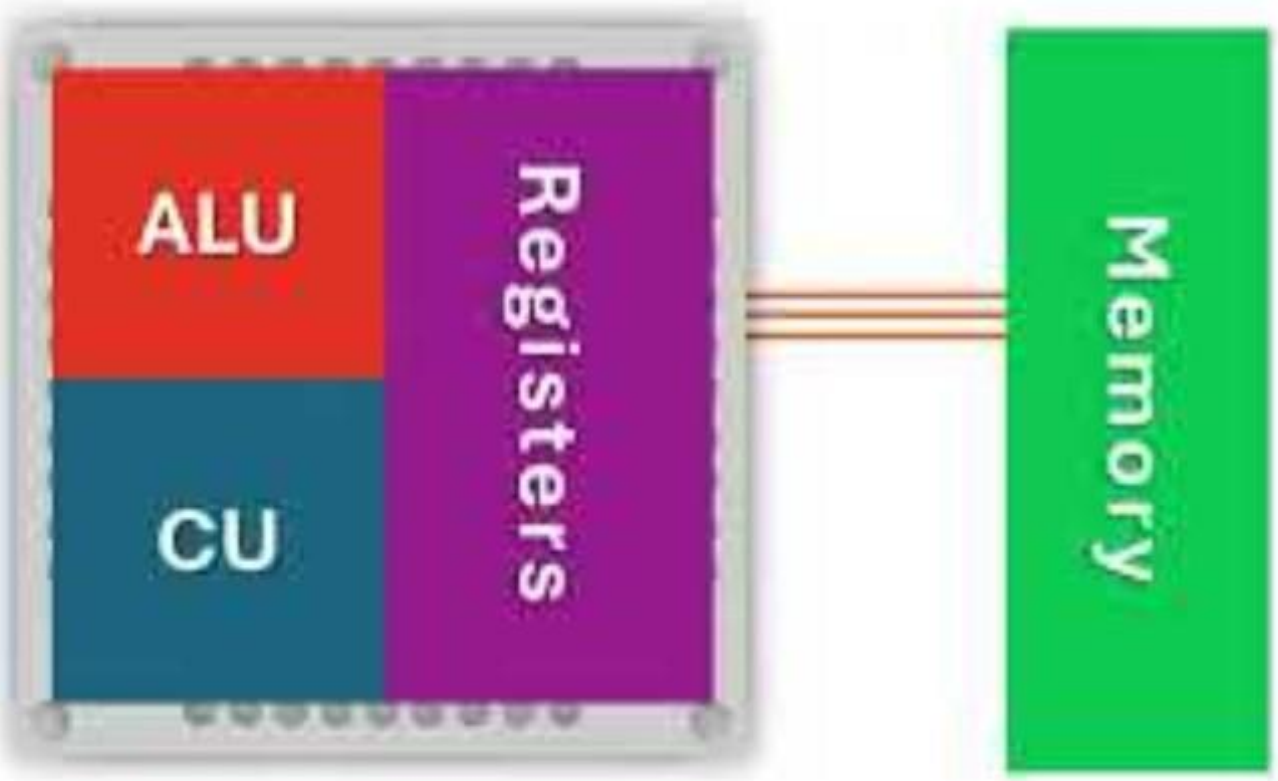
The Bus & Throughput

A bus is a high-speed electronic pathway (like wires or circuits) that acts as a shared communication system, allowing different components (CPU, memory, storage, graphics) to send data, address, and control signals to each other, functioning like a central data highway to transfer information **efficiently**.

Why do we need Buses?

1. Buses **simplify connection** by using shared channels, with different types of buses handling communication for internal and external devices.
2. Buses play an important role in minimizing the number of connections needed by centralizing communication over shared pathways.

Buses



Functional Classification: "What is moving?"

Every bus transaction requires three pieces of information. Imagine you are sending a letter: you need the content, the address, and the instructions (like "express" or "air-shipment").

1. **Data Bus**
   - Purpose: Carries the actual data (numbers, text, images).
   - Directional: Bidirectional. Data flows both into the CPU (reading) and out of the CPU (writing).
   - Capacity: The width (e.g., 64-bit) determines how much data can move at once. This means 64 separate wires carry 64 bits simultaneously in one clock cycle.
2. **Address Bus**
   - **Purpose:** Carries the location of where the data needs to go or come from in the memory.
   - **Direction:** Unidirectional**.** Usually, only the CPU sends addresses to tell memory, "I want the data at *this* specific spot."
   - **Capacity:** The width determines the maximum amount of RAM the computer can use (2^n addresses).
3. **Control Bus**
   - **Purpose:** Carries "command" signals to coordinate everything.
   - **Signals:** It tells the system if the current operation is a **Read** or a **Write**, handles **Interrupts** (signals from devices needing attention), and manages the **Clock** timing.

Structural Classification: "Where is it located?

This classification describes the hierarchy of the bus system – how close is the bus to the "heart" of the computer (the CPU).

1. **Local Bus** ("The Express Lane"): The local bus is the high-speed pathway directly connected to the CPU. It is designed for maximum speed and minimum delay.
   a. Connects: the CPU to the most critical, high-speed components like the L2/L3 cache or the integrated memory controller.
   b. Modern Context: In 2026, this is often handled by internal architectures like **Infinity Fabric** (AMD) or **Ring/Mesh Interconnects** (Intel).
2. **System Bus** ("The Main Road"): This is the primary backbone of a computer. Connects the CPU to the Main Memory (RAM).
3. **Peripheral/Expansion Bus**: This bus handles communication with **slower**, external devices. Connects the main system to things like your SSD, **GPU**, keyboard, or USB ports.
   a. Example: PCIe 6.0, USB4, and SATA.

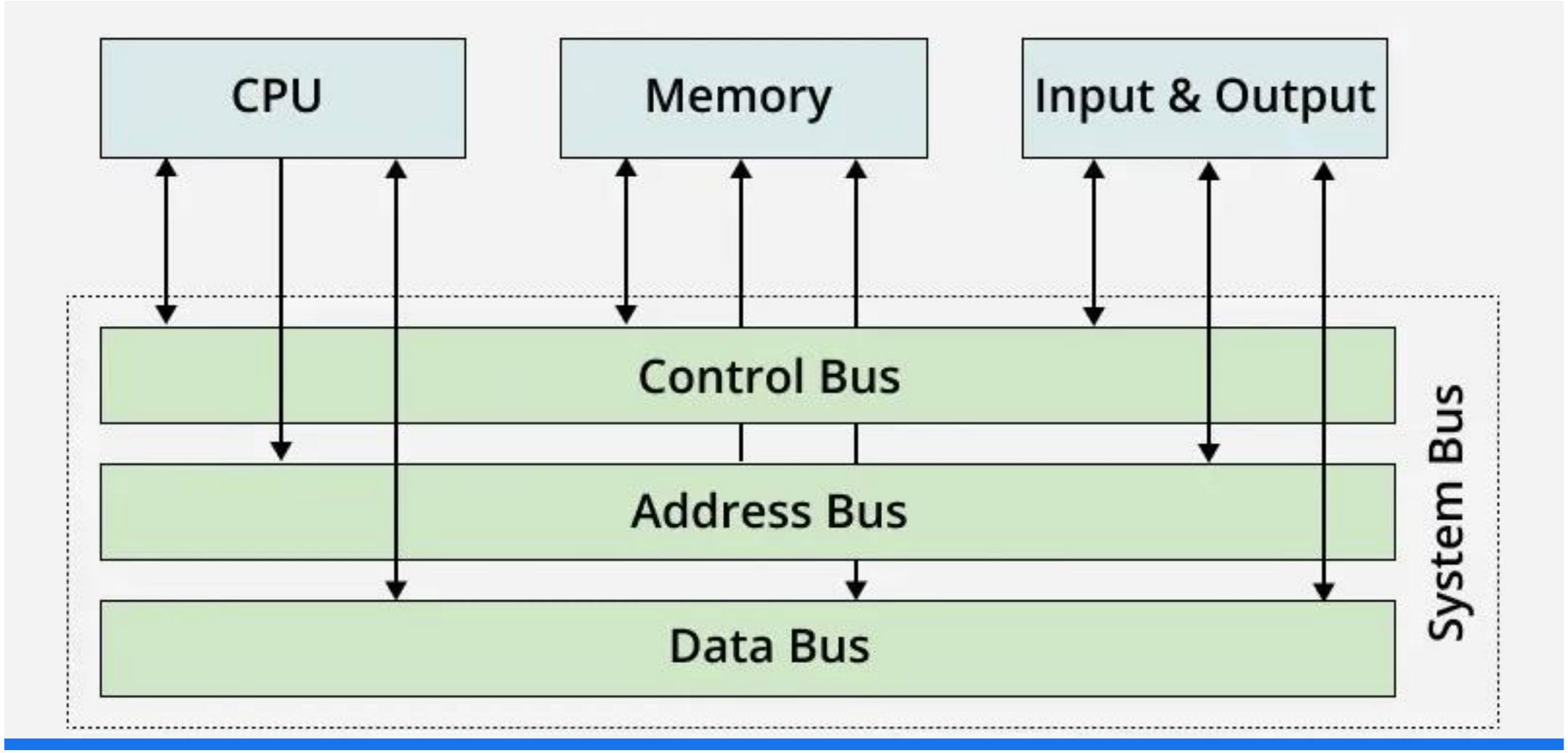
Local Bus vs. System Bus

The "Inside" and "Outside"

Historically, these were very different, but today the line is based on **physical boundaries**.

- **Local Bus (Internal):** This is the wiring **inside** the CPU chip itself. It connects the processing cores to the **Integrated Memory Controller (IMC)**. Since it's etched into the silicon, it is incredibly fast, short, and optimized for that specific chip.
- **System Bus (External):** Once the data leaves the CPU chip and travels across the motherboard to your **RAM sticks**, it is on the System Bus (often called the Memory Bus). These are the visible copper traces on your motherboard.

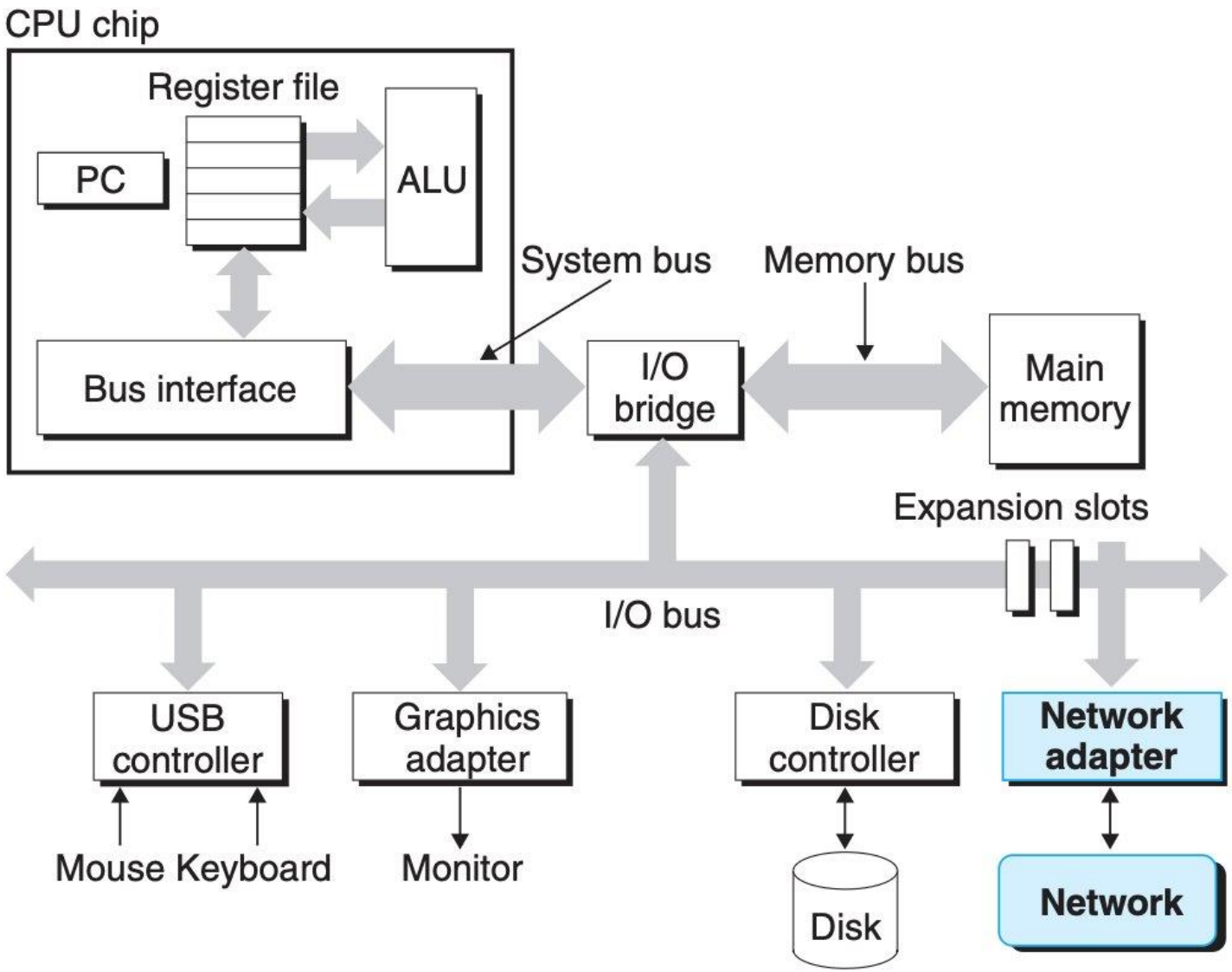
  The Difference: The Local Bus is like the internal plumbing of a house; the System Bus is the main water line on the street. One is private and ultra-fast; the other is a standardized connection to an external resource.


CPU
Memory
Input & Output
Control Bus
Address Bus
Data Bus
System Bus

How They Work Together

The "functional" buses (data, address, control) exist inside each "structural" bus.

Structural Bus	Contains...	Primary Job
Local Bus	Data, Address, Control	Moving data in/out of CPU registers at lightning speed.
System Bus	Data, Address, Control	Moving data between the CPU and the RAM.
Peripheral Bus	Data, Address, Control	Moving data between the CPU and the SSD or GPU.

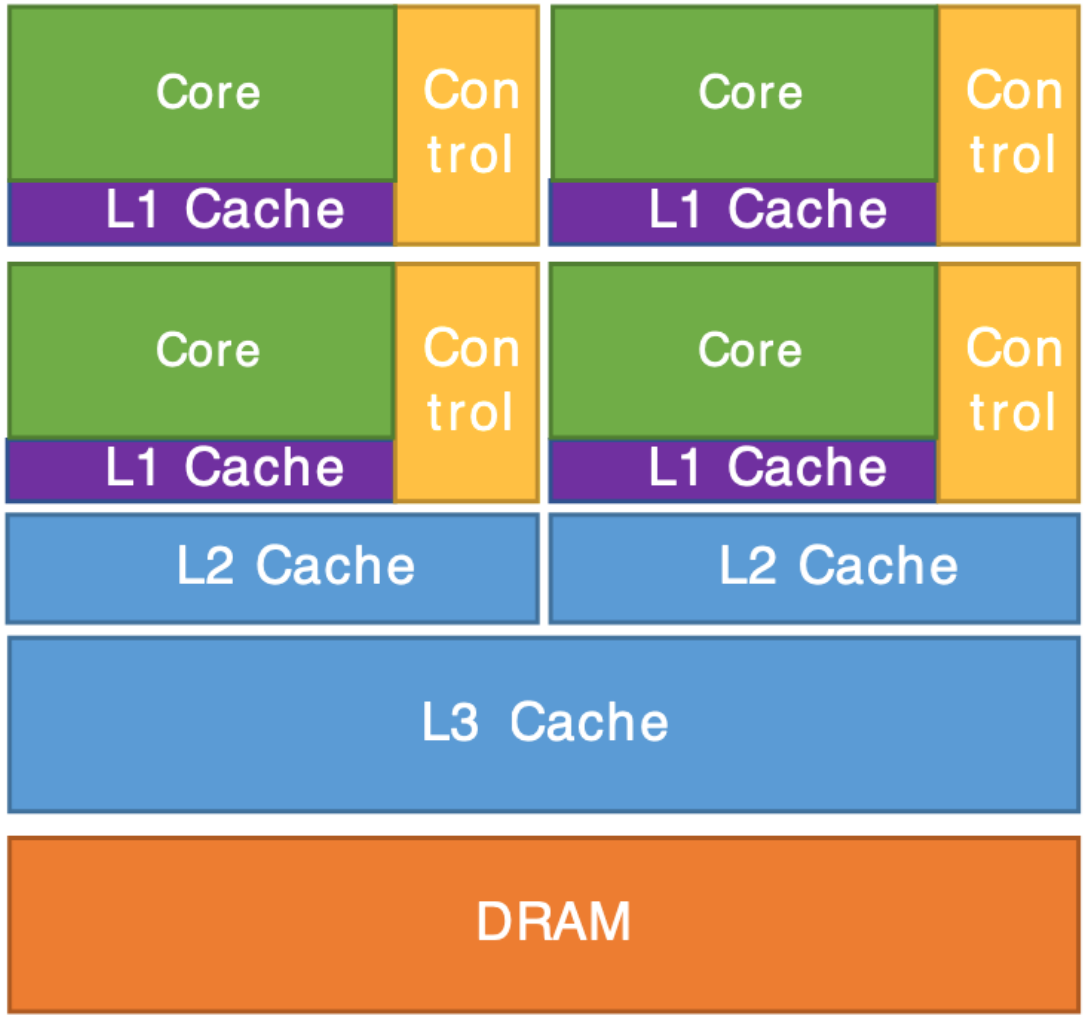

CPU chip
Register file
PC
ALU
Bus interface
System bus
Memory bus
I/O bridge
Main memory
Expansion slots
I/O bus
USB controller
Graphics adapter
Disk controller
Network adapter
Mouse Keyboard
Monitor
Disk
Network

GPU: Graphics Processing Unit

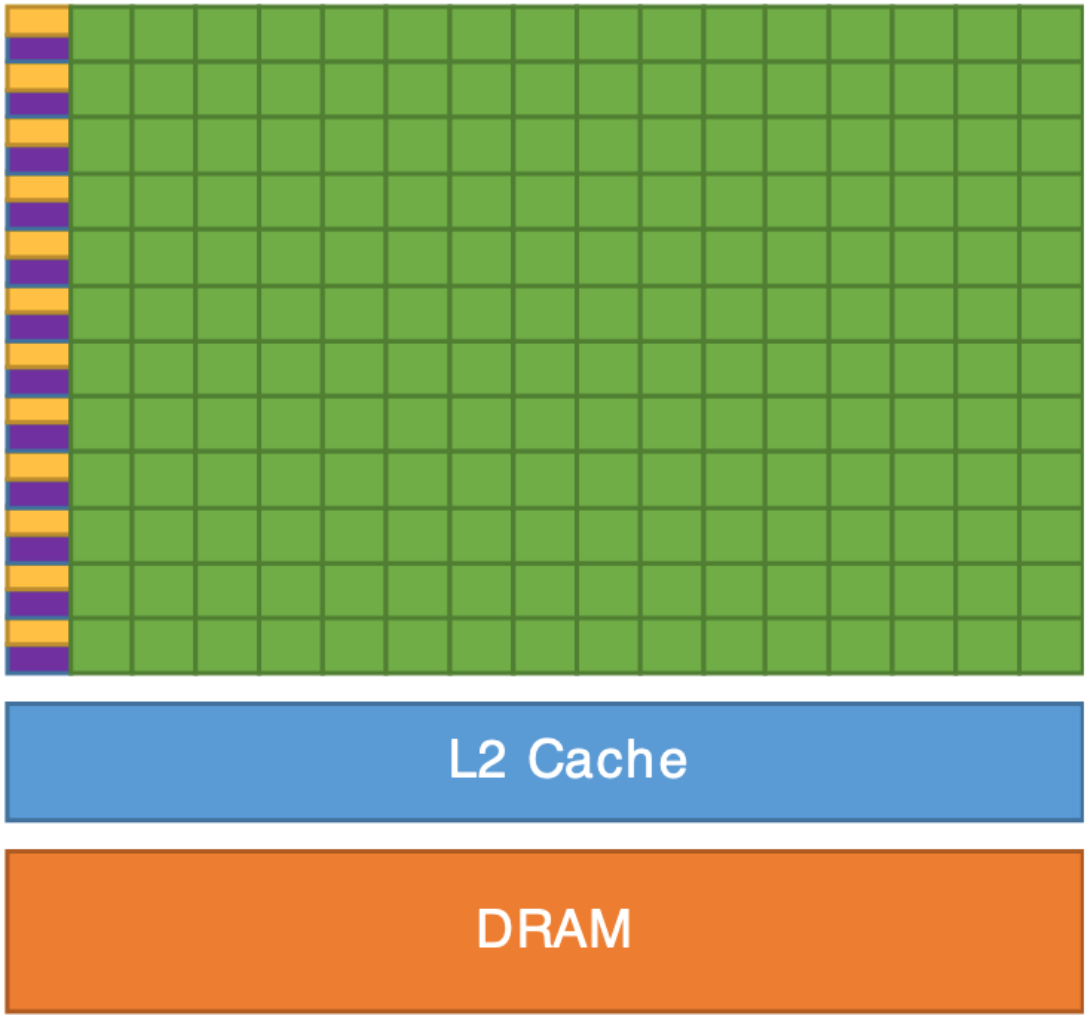
Question: Why is the GPU on a peripheral bus?
A peripheral bus (such as PCIe) is a standardized "language" that allows different brands to talk to each other. If GPUs used a "local bus", you could only use an Intel GPU with an Intel CPU.

GPUs are significantly faster than CPUs for tasks that can be broken down and processed in parallel, like AI, graphics, and scientific computing, due to their thousands of cores handling many simple calculations simultaneously, while CPUs, with fewer but more powerful cores, are faster for sequential, general-purpose tasks.

- **CPU:** General-purpose brain of the computer. Optimized for latency, *decision-making*, control flow, and diverse tasks.
- **GPU:** Specialized for high throughput *parallel* computation. Originally for graphics, now heavily used for AI, simulations, and data processing.


Core
Con trol
L1 Cache
Core
Con trol
L1 Cache
Core
Con trol
L1 Cache
Core
Con trol
L1 Cache
L2 Cache
L2 Cache
L3 Cache
DRAM

CPU


L2 Cache
DRAM

GPU

NVIDIA

Input/Output (I/O) Latency

To clearly understand the difference between a CPU cycle and a disk read, we must scale time up to human levels.

If one **CPU Cycle (0.3 nanoseconds)** was equal to **1 second**:

- **L1 Cache Access:** ~3 seconds.
- **Main Memory (RAM) Access:** ~6 minutes.
- **Solid State Drive (SSD) Read:** ~2 to 6 days.
- **Hard Drive (HDD) Seek:** ~1 to 12 months.
- **Internet Ping**: ~32 years.

This is why modern systems do everything possible to avoid "going to disk." A program that manages its data in RAM will always be exponentially faster than one that has to constantly read from a drive.

I/O Latency

Core idea: CPU time and I/O time live in different universes.

Time Scale Comparison

Event	Approx Time
CPU cycle	~0.3 ns
L1 cache hit	~1 ns
RAM access	~100 ns
NVMe SSD read	~100 µs
HDD seek	~5–10 ms

Disk I/O means reading or writing to the disk.

Why CPUs Don't Wait

Scenario: CPU speed: ~3 GHz (3 billion cycles per second)

Disk I/O Latency: milliseconds (millions of CPU cycles)

If CPU waited synchronously for Disk I/O:

- 3 GHz CPU → billions of CPU cycles wasted.
- CPU would sit idle doing nothing useful.
- Other programs could not run.
- UI would stop responding.

User Perception: "The system is frozen"

Why this is unacceptable?

Modern systems aim to:

1. Maximize CPU utilization.
2. Support multitasking.
3. Maintain responsiveness.
4. Scale to many processes.

Fundamental Design Principle

Blocking a fast component for a slow one violates all of these goals.

This principle appears across computer architecture, OS design, and distributed systems.

How to Solve for I/O Latency

Solutions

1. **Interrupts**: Device signals CPU when an operation completes.
    a. How it works: CPU issues disk request. CPU continues executing other tasks. Disk raises an interrupt when done. CPU briefly handles completion. Benefit: CPU stays productive.
2. **DMA**: Copying data byte-by-byte through the CPU is expensive.
    a. Disk controller transfers data directly to RAM. CPU is not involved in the data movement.
    b. CPU Role: Initiate transfer, handle completion interrupt.
3. **Asynchronous I/O**: I/O operations do not block the calling process. Instead, process issues I/O request. OS schedules other ready processes. Original process resumes later. Enables concurrency without wasting CPU time.

How to Solve for I/O Latency

4. Buffers & Caches: Temporary memory used to smooth speed differences. E.g., Disk writes collected in RAM buffer and written to disk in batches. Benefit: reduces the number of slow disk operations.

5. Caches: Fast memory storing frequently used data. Recently accessed disk blocks kept in RAM. Future reads avoid disk entirely.

How all Solutions Work Together - Typical disk read flow:

- Check cache (RAM)
- If miss, issue async disk request
- DMA transfers data to memory
- Interrupt signals completion
- Process resumes

CPU never waits idly.

Key Takeaways

1. CPU – disk speed gap is enormous.
2. Blocking leads to wasted cycles and frozen systems.
3. OS and Hardware collaborate to hide latency.
4. Interrupts, DMA, Async I/O, Buffers, and Caches are essential.

Performance is not about making slow things faster — it's about hiding their slowness.

Next Steps

Mid-Course Feedback: https://forms.gle/ADYhEevQD416Mhhm7

Tips to give effective feedback:

1. **Be specific and objective**: Avoid generalizations like "never" or "always".
2. **Make it actionable**: Focus on improvements that are within the person's control. Provide concrete suggestions or alternatives.
3. **Align with learning objectives**: Focus on whether the course met its stated goals and if the material helped you learn.
4. **Be constructive and respectful**: Maintain a polite and supportive tone, framing feedback as a tool for growth rather than purely a complaint.

Appendix

References

1. **Client-Server Architecture** is a system design where the client requests services or resources and the server provides them. Client initiates request (e.g., browser), servers responds (e.g., web server, database); clear separation of roles.
2. **Distributed Systems**: A distributed system is a system where multiple interconnected computers (nodes) work together over a network and appear as a single system to the user. No shared memory, communication over messages, and nodes can fail independently.
3. **Process Boundaries:** Logical or physical separation between running programs/processes. Crossing a boundary usually requires inter-process communication (IPC) or network calls.
4. **Communication Models**: Rules for how components exchange data. E.g., request-response, publish-subscribe, message queues, RPC.
5. **Fault Tolerance**: The ability of a system to continue operating despite of failures (node crashes, network issues, partial outages).

References

6. **Scalability Characteristics**: How well a system handles growth in load (users, data, requests). Includes vertical scaling (bigger machines) and horizontal scaling (more machines).

7. **MVC (Model-View-Controller):** Where model represents data and business logic, view means User Interface or presentation, and controller handles input, updates model and view.

8. **Layered Architecture Example**: Presentation Layer (UI), Application Layer (business logic), Data access layer, Database Layer. Each layer only interacts with the one below it.

9. **Event-Driven Systems:** Systems where components react to events instead of direct calls. Example: Order placed -> event emitted -> payment, inventory, and email services react.

10. **Module Boundaries** stands for clear division between components/modules that define what is inside the module, what is exposed via interfaces. Goals are isolation, clarity, and maintainability. **Dependency direction** provides the **allowed direction of reliance between modules.**

Difference between a CPU and a GPU

Aspect	CPU (Central Processing Unit)	GPU (Graphics Processing Unit)
Primary role	General-purpose computation	Massively parallel computation
Core count	Few (≈4–32 powerful cores)	Many (hundreds to thousands of simpler cores)
Core strength	Very strong, complex cores	Individually weak, simple cores
Parallelism	Limited parallelism	Extreme parallelism
Latency vs throughput	Optimized for **low latency**	Optimized for **high throughput**
Control flow	Excellent at branching and decision-heavy code	Poor at complex branching
Cache	Large, multi-level caches (L1/L2/L3)	Small cache per core

Difference between GPU and CPU

Aspect	CPU (Central Processing Unit)	GPU (Graphics Processing Unit)
Memory access	Optimized for random access	Optimized for sequential/batched access
Typical tasks	OS, applications, logic, control	Graphics, AI/ML, simulations, video processing
Programming style	Task-based, sequential + threads	Data-parallel (same operation on many data points)
Power efficiency	Efficient for varied workloads	Efficient for large, uniform workloads
Can replace the other?	Can run GPU tasks slowly	Cannot replace CPU for general computing